

Title: Strictly Chronological CNN Embeddings with Gradient-Boosted Trees for Next-Day Log-Return Forecasting

URL: <https://www.mdpi.com/2073-8994/18/3/416>

Source element: <article>

Open AccessArticle

Strictly Chronological CNN Embeddings with Gradient-Boosted Trees for Next-Day Log-Return Forecasting

by Zezhi Bao ^{1,†}, Xiaofei Li ^{1,*}, Menghuan Shi ¹, Yuen Huang ¹ and Junjie Du ²

¹

School of Information and Mathematics, Yangtze University, Jingzhou 434020, China

²

School of Information Engineering, Jingzhou University, Jingzhou 434020, China

*

Author to whom correspondence should be addressed.

†

These authors contributed equally to this work.

Symmetry 2026, 18(3), 416; <https://doi.org/10.3390/sym18030416>

Submission received: 18 January 2026 / Revised: 19 February 2026 / Accepted: 24 February 2026 / Published: 27 February 2026

(This article belongs to the Special Issue Symmetry in Artificial Intelligence and Applications)

Download

keyboard_arrow_down

Browse Figures Versions Notes

Abstract

Daily equity return forecasting is challenging due to low signal-to-noise ratios, heavy-tailed innovations, and persistent distribution drift. We study one-step-ahead log-return prediction using daily market variables and return-based transformations. We propose a CNN–LightGBM hybrid that transfers a last-step CNN embedding to a gradient-boosted tree regressor through explicit embedding standardization, which stabilizes the representation interface for tree learning. To reduce train-to-evaluation mismatch under drift, we adopt split-wise, training-only standardization with a recency-aware fit-latest-W rule. Return-related predictors are anchored on a one-sided wavelet-denoised close series, while other market channels are retained in their original form to preserve episodic extremes. Experiments on NIFTY50 with walk-forward model selection show statistically reliable accuracy gains over Naive0 and competitive performance against representative deep sequence baselines, and the supplementary evaluations on HDFC and INDA provide additional out-of-sample evidence on these two assets under the same strictly chronological protocol. A long-or-cash decision rule based on the return forecasts yields positive risk-adjusted performance under realistic transaction-cost assumptions, supporting the practical relevance of the predictive signal.

Keywords: log-return forecasting; CNN–LightGBM hybrid; embedding standardization; recency-aware standardization; fit-latest-W; wavelet denoising; economic value

1. Introduction

Forecasting equity movements remains a core challenge in quantitative finance [1,2]. Price dynamics are shaped by interacting drivers that range from macroeconomic policy and industry cycles to firm fundamentals, geopolitical shocks, and shifts in investor sentiment [3]. These interacting forces can create transient regularities, yet the resulting series are noisy and exhibit time-varying distributions, which makes reliable out-of-sample forecasting difficult [4].

Classical investment analysis is often organized around fundamental analysis and technical analysis [5]. Fundamental analysis evaluates intrinsic value through macro indicators, industry competition, and firm profitability [6]. Technical analysis summarizes market behavior using price–volume patterns and indicator families such as candlestick charts and moving averages [5,7]. Although widely used in practice, both approaches can be sensitive to modeling choices and to the timeliness and subjectivity of the information used [6,7].

Motivated by these limitations, a large body of work has explored statistical and machine-learning models for market prediction [8,9]. The Autoregressive Integrated Moving Average model, abbreviated as ARIMA, can be effective under approximate stationarity, but it often deteriorates when dynamics become nonlinear or regime dependent. The Generalized Autoregressive Conditional Heteroskedasticity model, abbreviated as GARCH, captures volatility clustering but can be less responsive around abrupt structural changes [10]. More recently, machine-learning models such as Support Vector Machines and Random Forests have been applied to improve predictive performance, while raising new questions regarding stability and interpretability under noisy labels [11].

Deep sequence models further expanded the modeling toolkit by learning representations directly from multi-variate histories [12]. CNN–LSTM variants are frequently used to combine local pattern extraction with temporal dependency modeling [13]. Hybrid designs that combine recurrent networks with gradient-boosted trees have also been reported in related financial tasks such as volatility forecasting [14]. In addition, wavelet-based denoising has been explored as a preprocessing component to improve the signal quality before a nonlinear predictor [15]. These studies support the view that combining complementary inductive biases can be useful in low signal-to-noise settings, provided that the experimental protocol is rigorously time-ordered [9,16]. However, many hybrid studies treat the handoff between a neural representation and a tree-based head as an implementation detail and provide limited specification on how scale stability is maintained across components. This under-specification makes it difficult to distinguish gains from an explicit representation-transfer mechanism from gains that could be reproduced by a trivial combination of standalone predictors. Our design addresses this issue by standardizing the transferred last-step embedding before tree regression and by validating the resulting gain against a simple averaging baseline and structured ablations.

Recent advances in time-series forecasting also include Transformer-style architectures [17]. Informer improves efficiency for long sequences through a sparse attention mechanism and a tailored decoding strategy [18]. TimesNet emphasizes multi-period structure by transforming one-dimensional series into richer representations and extracting patterns across frequency bands [19]. Related lines of work combine attention with feature engineering and heterogeneous inputs, including models that integrate auxiliary representations such as Time2Vec or multi-source indicator matrices [20,21]. While these architectures can be competitive, their reported gains can be sensitive to the forecasting target, the market under study, and the exact time-ordered evaluation protocol [17,22].

From the perspective of empirical forecasting, three methodological gaps remain salient [8,23]. First, inadvertent look-ahead can arise in feature construction and standardization when future statistics leak into model inputs or scaling, which can inflate backtest performance under nonstationary markets [16,24]. Second, hybrid models often emphasize architectural combination, yet the representation transfer across components is frequently under-specified, especially regarding scale stability when the downstream learner is sensitive to feature magnitude and temporal drift. Third, forecasting studies are frequently reported only through statistical accuracy metrics, whereas decision-oriented validation that maps forecasts to implementable actions is less consistently documented in the same strictly chronological setting [25].

To address these issues, we study one-step-ahead log-return prediction and propose a leakage-free CNN–LightGBM hybrid under a strictly chronological, no-look-ahead protocol. The inputs are constructed from market behavior variables and return-based transformations. To improve robustness under distribution shift, we adopt one-sided, past-only

standardization performed split-wise on the training segment, together with a recency-aware fit-latest-W strategy. We also incorporate wavelet denoising on the close price when forming return-related quantities to mitigate high-frequency noise. On the modeling side, a one-sided CNN encoder maps multivariate histories to a compact last-step embedding, and an explicit embedding standardization step is applied before the LightGBM regressor. The CNN encoder is used to learn a stable representation, and LightGBM performs the final nonlinear regression on the standardized embedding. The modular coupling retains the representation power of convolutional encoders while leveraging the efficiency of gradient-boosted decision trees for nonlinear interaction modeling. To verify that the improvement is not explained by naive hybridization, we report a simple averaging baseline and structured ablations under the same strictly chronological protocol, as summarized in Section 5.3.

The main contributions of this work are summarized as follows:

1.

A stability-oriented CNN–LightGBM hybrid with explicit representation transfer. We design a lightweight hybrid that couples a one-sided CNN encoder with a LightGBM regressor through a standardized last-step embedding, which stabilizes the feature interface for the downstream tree model.

2.

Train-only, recency-aware standardization for temporal shift. We implement a strictly chronological preprocessing and split-wise standardization protocol that uses training-only, past-only statistics. Within this protocol, we introduce a recency-aware fit-latest-W rule that estimates scaling statistics from the most recent portion of the training segment to better track evolving market conditions.

3.

Input-side robustness via one-sided wavelet denoising for return feature formation. When constructing return-based features, we replace the close price with its one-sided wavelet-denoised counterpart to mitigate high-frequency noise while preserving a past-only workflow.

4.

Evidence under a strictly chronological protocol. We conduct expanding walk-forward validation and a held-out test evaluation on NIFTY50, and we further examine robustness on HDFC and INDA. Performance is assessed against Naiveo and representative learned baselines with statistical significance testing, and we report Ensemble-Avg together with a structured ablation suite to rule out a trivial averaging explanation. We also complement error-based metrics with an economic evaluation that converts return forecasts into implementable long-or-cash decisions.

Hybrid forecasting models typically combine a sequential encoder with a downstream predictor in either an end-to-end manner or a loosely coupled two-stage pipeline. Representative designs such as CNN–LSTM fuse representations within a single neural system, while recurrent-network–tree hybrids often export intermediate neural states to a boosted-tree head with limited specification of interface normalization. This distinction matters because tree learners are sensitive to feature magnitude, and scale drift can change split behavior even when the upstream representation remains qualitatively similar.

Our CNN–LightGBM makes the coupling explicit through the last-step embedding and a training-only channel-wise embedding standardization applied before LightGBM. This design turns the handoff into a fixed and reproducible interface, rather than an implicit outcome of upstream training dynamics. In addition, the recency-aware fit-latest-W scaling rule is selected together with model hyperparameters under expanding walk-forward validation, which aligns normalization choices with the same strictly chronological selection objective as the model configuration. Together with the averaging baseline and structured ablations reported in Section 5.3, this positioning clarifies that the reported gains are linked to an explicit representation-transfer mechanism rather than a superficial hybrid composition.

The remainder of this paper is organized as follows. Section 2 specifies the proposed CNN-LightGBM framework and the role of each module. Section 3 details the strictly chronological data-processing protocol, including one-sided wavelet

denoising for return construction and training-only, split-wise standardization. Section 4 presents the experimental setup and the main empirical results, together with the corresponding visual diagnostics and method comparisons. Section 5 provides complementary analyses that assess robustness and interpretability, including sensitivity checks and evidence on how key design choices affect performance. Section 6 summarizes the findings and outlines directions for future work.

2. Proposed Model

This study proposes a cross-paradigm coupled temporal regression architecture that operates in tandem: a CNN-based feature encoder followed by a LightGBM predictor. The input consists of multivariate technical indicators stacked in chronological order. We apply split-wise standardization using training data only, with a fit-latest-W rule to reduce distribution drift while preserving a past-only protocol.

In the representation stage, the encoder begins with a

$$1 \times 1$$

pointwise convolution for channel mixing and to increase the embedding dimensionality, followed by BatchNorm, GELU, and SpatialDropout1D. We then stack a causal convolution (kernel size 3, dilation 1) with BatchNorm and GELU and connect it residually to the pointwise branch to form a causal residual block. For the CNN prediction head used during encoder training, we apply global average pooling and map the pooled features to an auxiliary output via a 48-unit dense layer and a linear head. The CNN is trained with mean squared error loss, using adaptive learning-rate decay and early stopping to curb overfitting.

During inference, we extract the embedding at the final time step from the convolutional feature maps and standardize it using training-set embedding statistics to remove cross-channel scale disparities.

Formal definition of embedding standardization.

Let

$$e_t \in \mathbb{R}^d$$

denote the last-step embedding extracted from the CNN convolutional feature maps at time t. Given the training segment

$$T_{\text{train}}$$

of each split, we compute the channel-wise mean

$$\mu_e = \frac{1}{N} \sum$$

$$t$$

$$\in$$

$$T$$

train

$$e$$

$$t$$

$$,$$

$$(1)$$

and define the channel-wise standard deviation in component form as

$$\sigma$$

$$e$$

$$,$$

$$j$$

$$=$$

$$1$$

$$N$$

$$\Sigma$$

$$t$$

$$\in$$

$$T$$

train

$$e$$

$$t$$

$$,$$

$$j$$

$$-$$

$$\mu$$

$$e$$

$$,$$

$$j$$

$$2$$

$$,$$

$$j$$

$$=$$

1

,

...

,

d

.

(2)

Channels with near-zero variance are removed using a training-only mask. We then obtain the standardized embedding

e

$\sim$

t

=

(

e

t

–

μ

e

)

$\oslash$

(

σ

e

+

ε

)

,

(3)

and apply the same

(

μ

e

,

σ

e

)

estimated on

T

train

unchanged to the corresponding validation and test segments.

Embedding standardization is a split-wise, fixed channel-wise centering and scaling transformation, together with a training-only variance mask. Its immediate effect is to normalize the numerical scale across embedding channels, which stabilizes the feature interface for the histogram-based discretization in LightGBM. Because the same fixed transformation is applied unchanged to the validation and test segments, this step does not perform any test-time adaptation and does not use evaluation-period statistics. Any improvement therefore reflects a more stable representation-transfer interface and a reduction of simple first- and second-moment mismatch between training and evaluation embeddings, rather than an adaptive regime-tracking mechanism.

On the one hand, the training objective of CNN is to minimize the prediction error, which does not constrain the output embedding to have a consistent numerical range across all channels. Therefore, significant scale discrepancies often exist among different channels, and a slight global level drift may occur over time. On the other hand, LightGBM adopts histogram-based discretization and approximate split search, where the valid candidate split points with limited samples are directly related to the numerical distribution observed in the training segment. When the scale discrepancies of embedding channels are excessively large or the distribution drifts over time, the discretization process will alter the downstream split selection and regularization behavior, thus introducing unnecessary interface sensitivity.

This embedding is then fed to a LightGBM regressor with

L

2

loss, and early stopping is triggered based on the validation MSE.

For hyperparameter optimization, we employ Optuna’s TPE to jointly search the fit-latest- W window W (WIN), the sequence look-back length L (LOOKBACK), the CNN learning rate

η

(LR), and key LightGBM hyperparameters. The optimization objective is a time-weighted walk-forward OOS-

R

2

against the strong baseline NaiveO, assigning larger weights to later validation folds to better match the test-set distribution. During training, we reserve a trailing slice of the training data as an internal validation set for CNN early stopping and for visualizing training dynamics, while the test set remains isolated throughout to ensure an objective assessment of out-of-sample performance.

By combining the CNNs ability to hierarchically extract temporal patterns with LightGBMs capacity to model complex, non-linear feature interactions, the resulting architecture exhibits stable out-of-sample performance under temporal evolution and distribution shift in our evaluated setting. Figure 1 and Figure 2 show the simplified overview of the research framework and the collaborative architecture details of CNN-LightGBM, respectively.

Figure 1. Diagrammatic illustration of the proposed research plan.

Figure 2. CNN–LightGBM hybrid framework. BN denotes batch normalization. GELU denotes the Gaussian error linear unit. SDrop1D denotes one-dimensional spatial dropout. k and d denote kernel size and dilation rate, respectively. GAP

denotes global average pooling. FC denotes a fully connected layer and 48 is the number of units. Train mean/std indicates embedding standardization using the training-set mean and standard deviation. MSE denotes mean squared error and L2 loss. Val MSE indicates that early stopping monitors the validation-set MSE. Feature sampling denotes LightGBM column subsampling. fit-latest-W denotes recency-aware, training-only standardization.

2.1. Convolutional Neural Network

The forecasting target is the next-day log return, and the predictive signal is weak relative to observation noise. In this regime, a compact encoder with a strong locality bias is attractive because it emphasizes short-horizon temporal motifs while limiting unnecessary capacity. We therefore use a one-sided temporal CNN to summarize the lookback window into a last-step embedding, and we delegate regime-dependent nonlinearities and cross-feature interactions to LightGBM. The empirical section evaluates LSTM-LightGBM and TimesNet under the same strictly chronological protocol, which provides an explicit comparison against recurrent and modern long-sequence alternatives.

One-sided temporal convolution and the no-look-ahead property.

Let

X

t

=

[

x

t

–

L

+

1

,

...

,

x

t

]

∈

R

L

×

p

denote the chronological lookback input at time t . We apply a one-dimensional one-sided convolution with kernel size

k

=

3

and dilation

d

=

1

. For a relative index

τ

$\in$

{

1

,

...

,

L

}

, the pre-activation output is

z

τ

=

Σ

i

=

0

k

–

1

W

i

x

τ

–

i

,

(4)

where

W

i

are trainable weights and the convolution uses no bias term. The activated feature is then computed by batch normalization followed by GELU,

y

τ

=

GELU

BN

z

τ

.

(5)

In implementation, one-sided left padding with

k

—

1

zeros is applied so that the output length remains L . As a result,

y

τ

depends only on

x

τ

,

x

τ

—

1

,

...

and never uses any input beyond the current index, which enforces the no-look-ahead property within the lookback window.

Encoder template and transferred embedding.

The encoder begins with a

1

×

1

pointwise convolution that mixes channels and maps inputs to an embedding dimension of

d

=

64

. This layer is followed by batch normalization, GELU, and one-dimensional spatial dropout with rate

0.10

. Next, we apply the one-sided convolution in Equation (4) and add it residually to the pointwise branch. A second one-dimensional spatial dropout with rate

0.05

is applied after the residual addition. This produces a sequence of feature maps of length L . The representation transferred to LightGBM is the last-step embedding

e

t

$\in$

R

64

extracted at the final index of the one-sided convolution feature map sequence.

During CNN training, we attach a lightweight regression head to provide a supervised signal. The head applies global average pooling over the feature map sequence, followed by a fully connected layer with 48 units and GELU activation, and a linear output trained with the mean squared error loss. For the proposed hybrid model, the final forecast is produced by LightGBM using the transferred and standardized embeddings. The CNN encoder specification is summarized in Table 1.

Table 1. CNN encoder specification.

2.2. Light Gradient Boosting Machine

LightGBM, an optimized gradient boosting framework, was developed with a focus on both efficiency and accuracy [26]. It addresses the need for a machine learning algorithm that can handle large-scale datasets with high-dimensional features more effectively than traditional gradient boosting methods [27,28].

The architecture of LightGBM is built upon several key innovations that enhance its performance. One of these is the histogram-based decision tree algorithm, which discretizes continuous features into histograms, thereby reducing both storage requirements and computational complexity. Another notable technique is Gradient-based One-Side Sampling (GOSS) [29,30], which optimizes the sampling process during training by focusing on instances with larger gradients. LightGBM also employs Exclusive Feature Bundling (EFB) [31,32], a method that bundles infrequent features together to further improve training efficiency.

LightGBM’s computation follows an iterative process of constructing decision trees. At each step, a new tree is added to the ensemble to correct the errors of the previous trees. This iterative process can be summarized as follows:

F

$$F_t = F_{t-1} + \lambda \cdot T_t \quad (6)$$

where

$$F_t$$

represents the updated prediction model after adding the t-th tree,

$$F_{t-1}$$

$$F_{t-1}$$

is the previous model,

$$T_t$$

is the new tree, and

$$\lambda$$

is a regularization parameter.

During tree construction, LightGBM adopts a leaf-wise growth strategy that selects, at each step, the leaf whose split yields the largest reduction in the objective function. Together with its histogram-based feature discretization and efficient sampling schemes, this design allows LightGBM to train high-accuracy models with lower computational cost and faster convergence than many conventional gradient boosting implementations [33].

In addition, LightGBM offers native support for parallel and distributed training, which allows it to scale effectively to large datasets and high-dimensional feature spaces. Overfitting is controlled through several regularization mechanisms, so that the learned model maintains strong predictive performance on unseen data. Taken together, these characteristics make LightGBM particularly suitable for machine learning tasks that involve large and structurally complex datasets [34,35].

In the iterative process of the eXtreme Gradient Boosting (XGBoost) algorithm, a level-wise leaf growth strategy is employed, where all leaf nodes at the same hierarchical level are split simultaneously, as illustrated in Figure 3a. However, this approach fails to differentiate the quality of leaves within the same level, forcing splits even on leaves with low gain values, which introduces substantial unnecessary computational overhead [36,37]. To address this, LightGBM implements a more efficient leaf-wise strategy. It iteratively selects the leaf with the highest splitting gain from all current leaves for division, cycling through this process (see Figure 3b). Compared to the level-wise approach, the leaf-wise method achieves greater error reduction and improved accuracy under the same number of splits [38,39]. Nevertheless, its potential drawback lies in generating deeper decision trees that may lead to overfitting. Therefore, LightGBM incorporates a maximum depth constraint to maintain efficiency while effectively mitigating overfitting risks [40].

Figure 3. Level -wise and leaf-wise tree growth [41].

3. Dataset Preparation

This section describes the data source and forecasting target, then reports stationarity diagnostics for the return series. It further details feature construction from price and trading activity, including denoising on the close price. Finally, it presents the leakage-free standardization and sequence construction used by the CNN–LightGBM pipeline, and summarizes all model inputs. The model is trained in the standardized domain and predictions are transformed back to the log-return domain for evaluation.

3.1. Data, Target, and Stationarity Tests

The empirical analysis uses the NIFTY 50 index with daily observations from January 2013 to December 2023. This period covers multiple market regimes, including the COVID-19 shock starting in 2020. We denote the open, high, low, and close prices by

O

t

,

H

t

,

L

t

, and

C

t

. Trading volume is denoted by

V

t

, and transaction value, or turnover, is denoted by

T

t

.

3.1.1. Forecasting Target (Log-Return)

Rather than modeling the non-stationary price level directly, we formulate the prediction task as one-step-ahead log-return forecasting. The supervised target is defined as

$$\begin{aligned}
 & r_{t+1} \\
 & = \log \left(\frac{C_{t+1}}{C_t} \right) - \log \left(\frac{C_t}{C_{t-1}} \right), \\
 & \tag{7}
 \end{aligned}$$

where

$$\begin{aligned}
 & r_{t+1} \\
 & + \\
 & 1 \\
 & \text{represents the next-day log-return. The main experiments focus on the one-day horizon (} \\
 & H \\
 & = \\
 & 1 \\
 & \text{).}
 \end{aligned}$$

3.1.2. Stationarity Diagnostics

To justify forecasting in the return domain, we conduct complementary stationarity tests on the price level (Close) and the one-day log-return series. Specifically, we apply the Augmented Dickey–Fuller (ADF) test, where the null hypothesis

is the presence of a unit root (non-stationarity), and the KPSS test, where the null hypothesis is level stationarity. Table 2 shows that the Close series fails to reject the unit-root null under ADF (

$$p$$

$$=$$

$$0.7981$$

) and rejects the stationarity null under KPSS (

$$p$$

$$=$$

$$0.0100$$

), indicating non-stationarity. In contrast, the log-return series strongly rejects the unit-root null under ADF (

$$p$$

$$=$$

$$0.0000$$

) and does not reject stationarity under KPSS (

$$p$$

$$=$$

$$0.1000$$

), indicating stationarity. These results align with the standard view that return forecasting removes the common trend in price levels and forces the model to learn market dynamics rather than the price scale. Accordingly, our task is formulated as next-day log-return prediction,

$$r$$

$$t$$

$$+$$

$$1$$

$$=$$

$$\log$$

$$($$

$$C$$

$$t$$

$$+$$

$$1$$

$$)$$

$$-$$

$$\log$$

$$($$

C

t

)

.

Table 2. ADF and KPSS stationarity test results for the Close series and log returns.

3.2. Feature Construction and Denoising

3.2.1. Feature Set for Next-Day Log-Return Forecasting

Based on the daily OHLCV records and transaction value (turnover), we construct a set of price- and volume-driven predictors tailored to one-step-ahead log-return forecasting. To reduce scale effects and improve numerical stability, we apply

log

(

1

+

.

)

transformations to trading activity, i.e.,

V

$\sim$

t

=

log

(

1

+

V

t

)

and

T

$\sim$

t

=

log

(

$$1 + T^t)$$

. To encode intraday price geometry in a scale-invariant manner, we further form relative log-price differences from

$$\left(\frac{O_t}{H_t}, \frac{L_t}{C_t} \right)$$

, including the log-range

$$\log \left(\frac{H_t}{L_t} \right) -$$

$$\log \left(\frac{L_t}{H_t} \right)$$

, the close-to-extrema gaps

$$\log \left(\frac{O_t}{C_t} \right)$$

H

t

)

–

log

(

C

t

)

and

log

(

L

t

)

–

log

(

C

t

)

, and the open-to-close change

log

(

O

t

)

–

log

(

C

t

)

. Finally, we include return-dynamics proxies computed from the close series, namely the current log return

r

t

=

log

(

C

t

)

–

log

(

C

t

–

1

)

,

(8)

together with

|

r

t

|

and

r

t

2

as volatility proxies, which help distinguish low- and high-volatility regimes.

All predictors are computed using information available up to time t only. In particular, no quantities involving

t

+

1

are used in feature construction. Therefore, the feature set does not introduce target leakage, and the evaluation strictly reflects out-of-sample forecasting of

r

t
+
1
.

3.2.2. Leakage-Free One-Sided Wavelet Denoising of the Close Price

Financial close prices often contain microstructure noise and short-lived fluctuations. These high-frequency components can destabilize return-based predictors that are computed from the close series, especially when the forecasting target is the next-day log return. To improve robustness while strictly avoiding look-ahead bias, we apply a one-sided, past-only wavelet denoising procedure to the close price and denote the resulting series as

C
 t
den
.

To avoid notation ambiguity, we denote the denoising window length by

L
 w

and reserve L for the sequence look-back length used in sample construction. The denoising is implemented using a sliding historical window. For each time t , we form a window of length

L
 w

using only past and present observations, namely

{
 C
 t
–
 L
 w
+
1
,
...
,
 C
 t
}

. Any missing values inside the window are imputed using only values within the same window, first by forward filling from earlier to later timestamps, then by a backward fill confined to the window. This step does not introduce future information beyond time t .

Given the windowed series, we perform a discrete wavelet transform and apply soft-thresholding to the detail coefficients. In the configuration selected in this study, we adopt a Daubechies wavelet of order four and determine thresholds using the BayesShrink rule. The threshold is determined by the BayesShrink rule, where both the noise level and the signal variance are estimated from the current historical window only.

Let d denote the finest-scale detail coefficients in the window. The noise standard deviation is estimated by the robust median estimator

$$\hat{\sigma} = \text{median} \left(\left| \frac{d}{0.6745} \right| \right) \tag{9}$$

Let $\hat{\sigma}_y^2$ be the empirical variance of the detail coefficients at a given scale. The underlying signal variance is then estimated as

$$\hat{\sigma}_x^2 = \max \left(\hat{\sigma}_y^2, \frac{\hat{\sigma}^2}{2} \right)$$

—

σ

$\wedge$

n

2

,

0

.